

09 — Development Architecture

Purpose

The purpose of this document is to define the development architecture of Mosaica.

This module establishes the principles, structures and practices required to develop, maintain and evolve the Mosaica software system.

The Development Architecture defines how the software will be created while preserving:

- maintainability,
- consistency,
- quality,
- collaboration capability,
- long-term evolution.

Development Architecture Scope

This module defines:

- development principles,
- repository organization,
- coding standards,
- testing approach,
- development workflows,
- continuous integration and deployment principles.

The purpose is not to define individual implementation details, but to establish a consistent development foundation.

Development Architecture Position

Development Architecture operates between architectural design and software implementation.

Product Architecture

↓

Development Architecture

↓

Source Code



Executable Application

Development Responsibilities

Code Organization

Responsible for:

- maintaining clear project structure,
 - separating responsibilities,
 - preserving architectural boundaries.
-

Development Standards

Responsible for:

- consistent coding practices,
 - readable implementation,
 - maintainable software.
-

Quality Assurance

Responsible for:

- testing practices,
 - defect prevention,
 - reliable releases.
-

Development Workflow

Responsible for:

- code changes,
- reviews,
- version management,
- release preparation.

Development Architecture Principles

Principle 1 — Architecture Before Implementation

Development decisions shall follow approved architectural decisions.

Code structure shall reflect:

- Domain boundaries,
- Application responsibilities,
- Infrastructure separation.

Principle 2 — Maintainability Over Short-Term Speed

Development choices shall prioritize long-term system health.

Short-term solutions that create future complexity shall be avoided.

Principle 3 — Consistent Structure

Similar problems should be solved using similar patterns.

The codebase should remain predictable for future developers.

Principle 4 — Quality Through Validation

Quality shall be supported through:

- testing,
- review processes,
- automated checks.

Principle 5 — Incremental Development

Mosaica shall evolve through controlled iterations.

Large uncontrolled changes shall be avoided.

Development Evolution Model

Mosaica development follows three phases.

Design

↓

Implementation

↓

Validation

↓

Release

Development Phase

Focus:

- implementing approved requirements,
 - preserving architecture,
 - creating maintainable code.
-

Validation Phase

Focus:

- testing behavior,
 - identifying defects,
 - confirming requirements.
-

Release Phase

Focus:

- controlled delivery,

- monitoring,
 - feedback collection.
-

Development Quality Goals

The Development Architecture aims to provide:

- understandable code,
 - predictable changes,
 - easier maintenance,
 - safer evolution,
 - reduced technical debt.
-

Module Structure

09 Development Architecture

09.01 Development Principles

09.02 Repository Structure

09.03 Coding Standards

09.04 Testing Architecture

09.05 CI/CD Approach

09.06 Development Architecture Review

Architectural Decisions Introduced

Decision 01

Development practices shall follow approved architectural boundaries.

Decision 02

Code quality and maintainability are long-term priorities.

Decision 03

Development complexity shall grow according to product requirements.

Decision 04

Testing and validation are part of development, not optional final steps.

09.01 — Development Principles

Purpose

The purpose of this section is to define the fundamental development principles that guide the implementation and evolution of Mosaica.

These principles establish the foundation for creating software that remains:

- maintainable,
- understandable,
- scalable,
- consistent,
- aligned with the approved architecture.

All development decisions shall be evaluated according to these principles.

Development Principles

Principle 1 — Architecture First

Definition

Development decisions shall follow the approved Mosaica architecture.

Implementation choices shall respect established boundaries between:

- Domain Layer,
- Application Layer,

- API Layer,
- Infrastructure Layer.

Application

Developers should not create solutions that bypass architectural responsibilities.

Example:

Incorrect:

API

↓

Database Update Directly

Correct:

API

↓

Application Service

↓

Domain Logic

↓

Database

Principle 2 — Clean Separation of Responsibilities

Definition

Each part of the system shall have a clear responsibility.

Application

The codebase should preserve separation between:

- user interaction,

- business workflows,
 - domain rules,
 - data access,
 - infrastructure concerns.
-

Reason

Clear boundaries reduce:

- complexity,
 - unexpected side effects,
 - maintenance difficulty.
-

Principle 3 — Readability Over Cleverness

Definition

Code should be written for human understanding.

Complex solutions shall not be preferred merely because they appear technically advanced.

Application

Prefer:

- clear names,
- simple structures,
- understandable logic.

Avoid:

- unnecessary abstractions,
 - overly complex patterns,
 - premature optimization.
-

Principle 4 — Small Incremental Changes

Definition

Development should progress through controlled, understandable changes.

Application

Changes should be:

- focused,
- testable,
- reviewable.

Large uncontrolled modifications should be avoided.

Principle 5 — Reuse With Purpose

Definition

Code reuse should reduce duplication without creating unnecessary complexity.

Application

Do not create reusable components before a real reuse pattern exists.

Avoid:

Create abstraction

↓

Maybe use later

Prefer:

Repeated need

↓

Create shared solution

Principle 6 — Domain Protection

Definition

Development shall preserve the Domain Model as the source of business truth.

Application

Business rules shall not be duplicated in:

- controllers,
 - database queries,
 - user interfaces.
-

Example

Incorrect:

Frontend decides:

Completed status is allowed

Correct:

Library Entry Aggregate

↓

Validates status transition

Principle 7 — Quality Is Continuous

Definition

Quality practices are part of normal development.

They are not activities performed only before release.

Application

Quality includes:

- testing,
- code review,
- validation,

- documentation.
-

Principle 8 — Avoid Premature Complexity

Definition

Development solutions shall match current requirements.

Application

Avoid introducing:

- unnecessary frameworks,
 - complex architecture patterns,
 - unused infrastructure.
-

Principle 9 — Documentation as Part of Development

Definition

Important technical decisions shall be documented.

Application

Documentation should explain:

- why a decision exists,
 - what problem it solves,
 - what limitations it has.
-

Development Decision Model

Development Decision

↓

Does it follow architecture?

↓

Does it reduce complexity?

↓

Does it improve maintainability?

↓

Implement

Development Workflow Model

Requirement

↓

Design

↓

Implementation

↓

Validation

↓

Integration

↓

Release

Development Quality Goals

Mosaica development should achieve:

- understandable codebase,

- predictable changes,
 - minimal technical debt,
 - sustainable growth.
-

Architectural Decisions Confirmed

Decision 01

Code implementation must follow approved architecture.

Decision 02

Domain rules remain protected from implementation leakage.

Decision 03

Simple and understandable solutions are preferred.

Decision 04

Development progresses through controlled iterations.

Decision 05

Complexity requires justification.

09.02 — Repository Structure

Purpose

The purpose of this section is to define the repository structure of Mosaica.

This section establishes how source code, documentation and development resources are organized within the project repository.

The objective is to create a predictable structure that supports:

- maintainability,
- architectural clarity,
- collaboration,
- future expansion.

Repository Structure Principles

Principle 1 — Architecture Reflected in Structure

Definition

The repository structure shall represent the approved system architecture.

Application

The code organization should make architectural boundaries visible.

Example:

Repository

├─ API

├─ Application

├─ Domain

└─ Infrastructure

Principle 2 — Clear Ownership

Definition

Each folder and module shall have a clearly defined responsibility.

Application

A component should have one primary reason to exist.

Unclear ownership should be avoided.

Principle 3 — Separation of Code and Documentation

Definition

Technical documentation shall remain organized separately from implementation code.

Repository Example

Mosaica Repository

```
|— src
|— tests
|— docs
|— scripts
└─ configuration
```

Repository Root Structure

The Mosaica repository follows this high-level structure:

Mosaica

```
|— src
|— tests
|— docs
```

```
|— assets
|— scripts
|— configuration
└— README
```

src Directory

Purpose

Contains application source code.

Structure

```
src
|— API
|— Application
|— Domain
|— Infrastructure
└— Shared
```

API Directory

Responsibility

Contains external communication layer.

Includes:

- controllers,
 - request handling,
 - response formatting.
-

Does Not Contain

- business rules,

- database operations,
 - domain decisions.
-

Application Directory

Responsibility

Contains application workflows.

Includes:

- Application Services,
 - use case coordination,
 - workflow orchestration.
-

Does Not Contain

- core business rules,
 - direct database manipulation.
-

Domain Directory

Responsibility

Contains business logic.

Includes:

- Aggregates,
 - Entities,
 - Value Objects,
 - Domain Services.
-

Protection Rule

Domain should remain independent from external technical concerns.

Infrastructure Directory

Responsibility

Contains technical implementations.

Includes:

- database access,
 - external service integrations,
 - infrastructure components.
-

Shared Directory

Purpose

Contains truly shared technical components.

Rule

Shared code should only exist when a genuine cross-module requirement exists.

tests Directory

Purpose

Contains automated validation.

Structure

tests

├─ Unit

├─ Integration

└─ EndToEnd

Unit Tests

Purpose:

Validate isolated components.

Integration Tests

Purpose:

Validate communication between system components.

End-To-End Tests

Purpose:

Validate complete user workflows.

docs Directory

Purpose

Contains project documentation.

Examples

docs

└─ Architecture

└─ ADR

└─ API

└─ Database

└─ Guides

assets Directory

Purpose

Contains project resources.

Examples:

- images,
 - design resources,
 - static files.
-

scripts Directory

Purpose

Contains development automation scripts.

Examples:

- setup scripts,
 - migration scripts,
 - utility tools.
-

configuration Directory

Purpose

Contains environment and project configuration references.

Rule

Sensitive credentials shall not be stored directly in the repository.

Repository Evolution

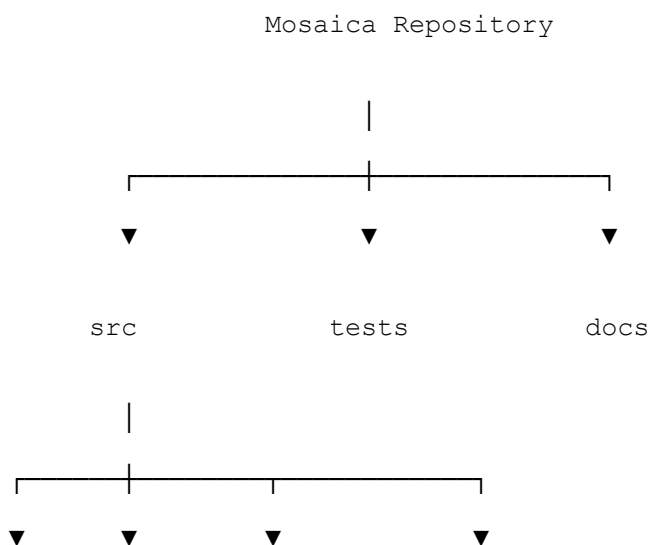
The repository structure shall evolve gradually.

Initial structure should prioritize:

- clarity,
- simplicity,
- maintainability.

Additional modules shall only be introduced when justified.

Repository Structure Model



API Application Domain Infrastructure

Architectural Decisions Confirmed

Decision 01

Repository structure shall reflect system architecture.

Decision 02

Domain, Application and Infrastructure responsibilities remain separated.

Decision 03

Documentation is maintained as a first-class project resource.

Decision 04

Shared components require a clear justification.

Decision 05

Repository complexity grows only when project needs require it.

09.03 — Coding Standards

Purpose

The purpose of this section is to define the coding standards that guide the implementation of Mosaica.

These standards establish consistent practices for:

- code readability,
- maintainability,
- quality,
- architectural alignment.

The objective is to ensure that the codebase remains understandable and sustainable as Mosaica evolves.

Coding Standard Principles

Principle 1 — Readability First

Definition

Code shall be written primarily for human understanding.

Readable code is preferred over shorter or more clever solutions.

Application

Developers should prefer:

- meaningful names,
- clear structures,
- simple logic.

Avoid:

- unexplained abbreviations,
 - unnecessary complexity,
 - difficult-to-follow shortcuts.
-

Principle 2 — Consistent Naming

Definition

Naming conventions shall be consistent throughout the codebase.

Names should clearly communicate purpose.

Application

Examples:

Good:

CreateLibraryEntryService

LibraryEntry

RecordConsumptionEvent

Avoid:

CreateLE

DoStuff()

Principle 3 — Single Responsibility

Definition

Each component should have one clear responsibility.

Application

A class or module should not combine unrelated concerns.

Example:

Incorrect:

```
LibraryService
```

- create library entry
 - send emails
 - manage database connection
 - generate reports
-

Correct:

```
LibraryEntryService
```

```
NotificationService
```

```
ReportService
```

Principle 4 — Respect Architectural Boundaries

Definition

Code shall remain inside its designated architectural layer.

Application

API Layer:

Responsible for communication.

Application Layer:

Responsible for workflows.

Domain Layer:

Responsible for business rules.

Infrastructure Layer:

Responsible for technical implementations.

Rule

A lower-level technical concern shall not leak into higher-level business logic.

Principle 5 — Domain Logic Protection

Definition

Business rules shall exist only inside the Domain Layer.

Application

Avoid:

Controller

↓

Business Rule Calculation

Prefer:

Application Service

↓

Domain Aggregate

↓

Business Decision

Principle 6 — Avoid Premature Abstraction

Definition

Abstractions should solve existing problems.

They should not be created only for possible future needs.

Application

Avoid:

Create generic system

↓

Maybe useful later

Prefer:

Real repeated need

↓

Create abstraction

Principle 7 — Error Handling Consistency

Definition

Errors shall be handled predictably throughout the application.

Application

Error handling should:

- provide meaningful information,

- avoid exposing sensitive details,
 - preserve system integrity.
-

Principle 8 — Documentation of Important Decisions

Definition

Important technical decisions shall be documented.

Application

Documentation should explain:

- why a decision was made,
 - what alternatives were considered,
 - what limitations exist.
-

Principle 9 — Code Duplication Control

Definition

Unnecessary duplication should be avoided.

However, forced abstraction should also be avoided.

Balance

The goal is:

Less duplication

+

Clear responsibility

=

Maintainable code

Principle 10 — Security-Aware Development

Definition

Development practices shall respect the Security Architecture.

Application

Developers must consider:

- data exposure,
 - authorization,
 - secure handling,
 - sensitive information protection.
-

Code Review Principles

Code review should evaluate:

- architectural alignment,
 - readability,
 - correctness,
 - maintainability,
 - security impact.
-

Development Quality Checklist

Before accepting code:

Does it follow architecture?

↓

Is responsibility clear?

↓

Is it understandable?

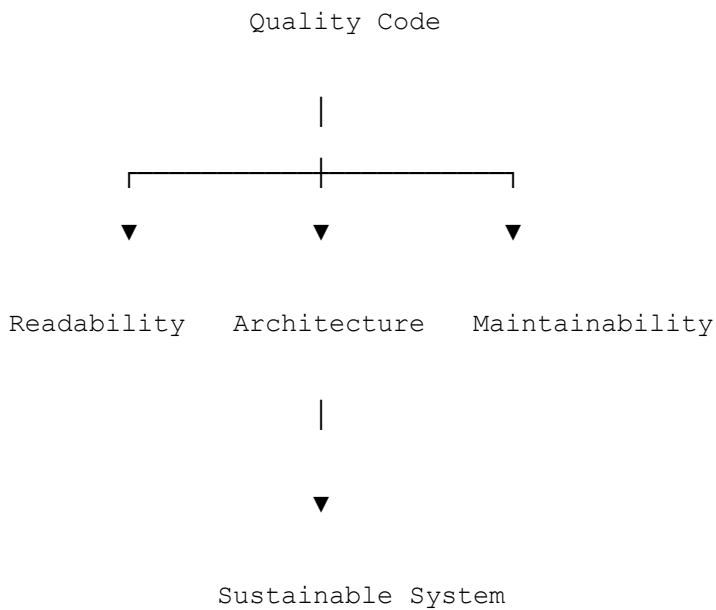
↓

Is it tested?

↓

Does it introduce unnecessary complexity?

Coding Standards Model



Architectural Decisions Confirmed

Decision 01

Readable code is preferred over clever code.

Decision 02

Code structure must respect architectural boundaries.

Decision 03

Business rules remain protected inside the Domain Layer.

Decision 04

Abstractions require real justification.

Decision 05

Security and maintainability are development responsibilities.

09.04 — Testing Architecture

Purpose

The purpose of this section is to define the testing architecture of Mosaica.

This section establishes the principles, testing layers and validation approach required to maintain software quality throughout development.

The objective is to ensure that Mosaica remains reliable, maintainable and aligned with its approved architecture.

Testing Principles

Principle 1 — Quality Is Continuous

Definition

Testing shall be part of normal development activities.

Testing shall not be treated as a final step before release.

Application

Validation should occur throughout:

Development

↓

Testing

↓

Integration

↓

Release

Principle 2 — Test According to Risk

Definition

Testing effort shall focus on areas where failures create the highest impact.

Application

Priority areas:

- Domain Rules,
 - User-owned data,
 - Application workflows,
 - Security boundaries.
-

Principle 3 — Domain Logic Requires Strong Validation

Definition

Business-critical behavior shall receive the highest level of testing.

Application

Examples:

- Library Entry status changes,
- Consumption Event creation,
- ownership rules,
- business validations.

Principle 4 — Tests Should Protect Behavior

Definition

Tests should verify what the system does, not how the code happens to be implemented.

Application

Prefer:

User adds Content to Library

↓

Expected business result

Over:

Specific internal method execution

Testing Layer Model

Mosaica testing consists of three primary levels.

Testing Architecture

├─ Unit Testing

├─ Integration Testing

└─ End-to-End Testing

1. Unit Testing

Purpose

Validates individual components in isolation.

Primary Targets

Unit tests should focus on:

- Entities,
 - Value Objects,
 - Domain Services,
 - Application logic components.
-

Examples

`Library Entry`

↓

`Can status change from Planned to Completed?`

`Consumption Event`

↓

`Can event be added correctly?`

Responsibility

Unit tests verify that individual pieces behave according to expected rules.

2. Integration Testing

Purpose

Validates communication between system components.

Primary Targets

Integration tests should verify:

- Application Services,
- Repositories,
- Database communication,
- External service boundaries.

Examples

CreateLibraryEntryService

↓

Library Entry Repository

↓

Database Persistence

3. End-to-End Testing

Purpose

Validates complete user workflows.

Primary Targets

Important user scenarios.

Examples

User Login

↓

Search Content

↓

Add To Library

↓

Update Status

↓

Record Consumption

Testing Responsibility by Layer

Layer	Testing Focus
API Layer	Request and response behavior
Application Layer	Workflow execution
Domain Layer	Business rules
Infrastructure Layer	Technical integration

Domain Testing Priority

The Domain Layer receives special attention because it contains business truth.

Priority areas:

Domain Testing

- └─ Aggregate Rules
 - └─ Entity Behavior
 - └─ Value Object Validation
 - └─ Domain Services
-

Test Data Principles

Definition

Test data shall be controlled and understandable.

Rules

Test data should:

- represent realistic scenarios,
- avoid unnecessary complexity,

- not contain real User information.
-

Testing and Security

Tests shall respect security principles.

Requirements

Testing should verify:

- authorization boundaries,
 - ownership protection,
 - sensitive data handling.
-

Testing Evolution Strategy

Testing maturity grows with product maturity.

Personal Phase

↓

Growing Phase

↓

Product Phase

Personal Phase

Focus:

- important unit tests,
 - critical workflows,
 - manual validation where appropriate.
-

Product Phase

Focus:

- expanded automation,
 - regression testing,
 - continuous validation.
-

Testing Decision Model

New Feature

↓

What can fail?

↓

Which layer owns this behavior?

↓

Add appropriate test

↓

Validate Change

Architectural Decisions Confirmed

Decision 01

Testing is part of development, not a final activity.

Decision 02

Domain behavior receives priority testing attention.

Decision 03

Tests validate behavior rather than implementation details.

Decision 04

Testing complexity grows with product maturity.

Decision 05

Security boundaries shall be included in validation.

09.05 — CI/CD Approach

Purpose

The purpose of this section is to define the Continuous Integration and Continuous Deployment approach of Mosaica.

This section establishes how code changes are validated, integrated and released while maintaining software quality and architectural consistency.

The objective is to create a reliable development workflow that reduces errors and supports controlled evolution.

CI/CD Principles

Principle 1 — Automated Validation

Definition

Code changes should be automatically validated before integration.

Application

Validation may include:

- build checks,
- automated tests,
- code quality checks.

Principle 2 — Small and Controlled Changes

Definition

Development changes should be integrated through small, understandable updates.

Application

Avoid:

Large change

↓

Unknown problems

↓

Difficult debugging

Prefer:

Small change

↓

Validate

↓

Integrate

Principle 3 — Quality Before Release

Definition

A release should only occur after required validation steps are completed.

Application

A successful build does not automatically mean a release is ready.

Quality checks must be considered.

Principle 4 — Repeatable Deployment

Definition

Deployment processes should be predictable and reproducible.

Application

The same process should be used consistently between environments.

CI/CD Pipeline Model

Mosaica follows this general workflow:

Code Change

↓

Version Control

↓

Build

↓

Automated Validation

↓

Review

↓

Deployment

Continuous Integration

Purpose

Ensures that new code changes integrate safely with the existing system.

CI Responsibilities

CI process should verify:

- code builds successfully,
 - tests pass,
 - basic quality requirements are satisfied.
-

Continuous Deployment

Purpose

Ensures controlled delivery of validated changes.

Deployment Responsibility

Deployment process manages:

- application packaging,
 - environment delivery,
 - release execution.
-

Development Workflow

Mosaica development follows this flow:

Feature Requirement

↓

Implementation

↓

Local Validation

↓

Integration

↓

Testing Environment

↓

Production Release

Version Control Principles

Principle

All important code changes shall be tracked through version control.

Requirements

Version control should support:

- change history,
 - collaboration,
 - rollback capability.
-

Branching Approach

Mosaica uses a simple branching strategy.

Initial approach:

Main Branch

↑

Feature Changes

Main Branch

Purpose:

Contains stable code.

Feature Changes

Purpose:

Contains development work before integration.

Release Management

Releases should be identifiable and traceable.

Release Information

A release should communicate:

- what changed,
 - why it changed,
 - whether validation passed.
-

Rollback Strategy

Principle

Failed releases should be reversible.

Purpose

Rollback protects:

- system availability,
 - User experience,
 - data integrity.
-

CI/CD Evolution Strategy

The CI/CD process grows with project maturity.

Personal Phase

↓

Growing Phase

↓

Product Phase

Personal Phase

Focus:

- basic version control,
 - manual deployment when appropriate,
 - essential validation.
-

Product Phase

Focus:

- automated pipelines,
 - release automation,
 - deployment monitoring.
-

CI/CD Decision Model

Code Change

↓

Validate

↓

Review

↓

Integrate

↓

Release

Architectural Decisions Confirmed

Decision 01

Code changes require validation before release.

Decision 02

CI/CD complexity shall match project maturity.

Decision 03

Deployment processes should be repeatable.

Decision 04

Version control is required for controlled evolution.

Decision 05

Failed releases should have recovery options.

09.06 — Development Architecture Review

Review Objective

The purpose of this Architecture Review is to validate that the Mosaica Development Architecture provides a consistent and maintainable foundation for software implementation.

This review confirms that development practices preserve:

- architectural boundaries,
- code quality,

- testing discipline,
- controlled evolution,
- long-term maintainability.

Scope Reviewed

The following development architecture components have been reviewed:

- Development Principles
- Repository Structure
- Coding Standards
- Testing Architecture
- CI/CD Approach

Validation Results

Architecture Alignment

Status: Approved

The Development Architecture correctly reflects the approved Mosaica architecture.

Development practices preserve separation between:

API Layer

↓

Application Layer

↓

Domain Layer

↓

Infrastructure Layer

Repository Structure

Status: Approved

The repository structure correctly represents system responsibilities.

Confirmed:

- source code is separated by architectural responsibility,
 - documentation remains independent from implementation,
 - tests have a dedicated structure.
-

Coding Standards

Status: Approved

Coding standards establish a maintainable development approach.

Confirmed principles:

- readability over cleverness,
 - clear naming,
 - single responsibility,
 - avoidance of unnecessary abstraction.
-

Domain Protection

Status: Approved

Development rules correctly protect Domain responsibilities.

Confirmed:

- business rules remain inside Domain Layer,
 - technical concerns do not leak into business logic,
 - Application Layer coordinates instead of replacing Domain behavior.
-

Testing Architecture

Status: Approved

The testing approach correctly prioritizes important system behavior.

Confirmed testing focus:

- Domain rules,

- Application workflows,
 - security boundaries,
 - critical User scenarios.
-

CI/CD Approach

Status: Approved

The CI/CD approach provides controlled software evolution without unnecessary operational complexity.

Confirmed:

- changes are validated,
 - releases are controlled,
 - deployment can evolve gradually.
-

Development Complexity Evaluation

Status: Approved

The Development Architecture follows the principle:

Development complexity shall grow according to actual product requirements.

The architecture avoids premature introduction of:

- excessive automation,
 - unnecessary tooling,
 - complex workflows.
-

Personal Development Compatibility

Status: Approved

The development approach supports Mosaica's initial personal usage phase.

Confirmed:

- development can begin without expensive tooling,
- local development remains possible,
- professional practices can be introduced gradually.

Architectural Decisions Confirmed

Decision 01

Development practices shall follow approved architectural boundaries.

Decision 02

Repository structure shall reflect system architecture.

Decision 03

Code quality depends on consistency and maintainability.

Decision 04

Testing shall protect critical system behavior.

Decision 05

Automation shall increase according to project maturity.

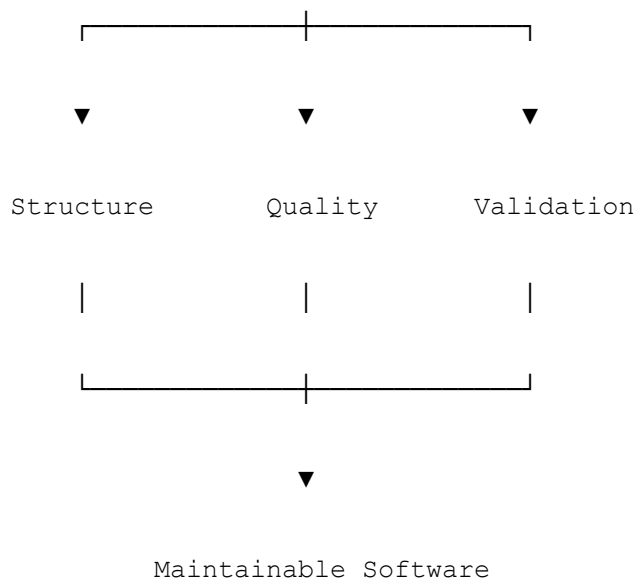
Decision 06

Development workflows shall support both personal and future product phases.

Development Architecture Summary

Development

|



Review Conclusion

The Mosaica Development Architecture is considered architecturally complete.

The current design provides:

- a clear development structure,
- maintainable coding practices,
- controlled quality processes,
- a gradual path from personal development to future product development.

No blocking architectural issues have been identified.